
# CE 222 – *Computer Organization and Assembly Language (COAL)*
# Lecture 10

Dr. Asad Mahmood

Feb. 09, 2026

# *Lecture Outline*

- Announcements (Assignment and Quiz 2 Day/Time/Venue)
- Outline of Previous Lecture:
  - **Chapter 2 - Instructions: Language of the Computer**
    - 2.6 Logical Operations
    - 2.7 Instructions for Making Decisions

- Outline to Today's Lecture:
  - **Chapter 2 - Instructions: Language of the Computer**
    - 2.7 Instructions for Making Decisions
    - 2.8 Supporting Procedures in Computer Hardware

# Chapter 2

## Instructions: Language of the Computer

# Chapter Outline

- **2 Instructions: Language of the Computer**
  - 2.1 Introduction
  - 2.2 Operations of the Computer Hardware
  - 2.3 Operands of the Computer Hardware
  - 2.4 Signed and Unsigned Numbers
  - 2.5 Representing Instructions in the Computer
  - 2.6 Logical Operations
  - 2.7 Instructions for Making Decisions
  - 2.8 Supporting Procedures in Computer Hardware
  - 2.9 Communicating with People
  - 2.10 RISC-V Addressing for Wide Immediates and Addresses
  - 2.11 Parallelism and Instructions: Synchronization
  - 2.12 Translating and Starting a Program
  - 2.13 A C Sort Example to Put it All Together
  - 2.14 to 2.24 …..

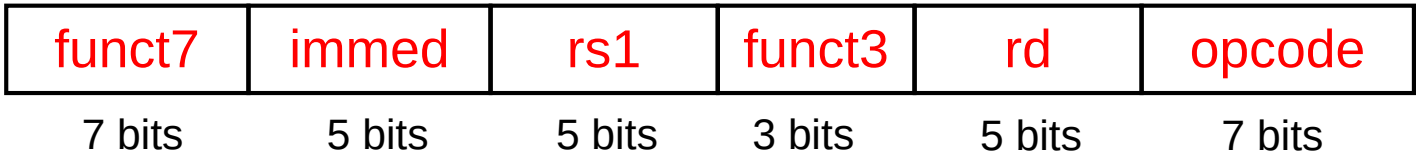
# Logical Operations

- Instructions for bitwise manipulation

| Operation | C | Java | RISC-V |
|---|---|---|---|
| Shift left | << | << | slli |
| Shift right | >> | >>> | srli |
| Bit-by-bit AND | & | & | and, andi |
| Bit-by-bit OR | \| | \| | or, ori |
| Bit-by-bit XOR | ^ | ^ | xor, xori |
| Bit-by-bit NOT | ~ | ~ | |

- Useful for extracting and inserting groups of bits in a word

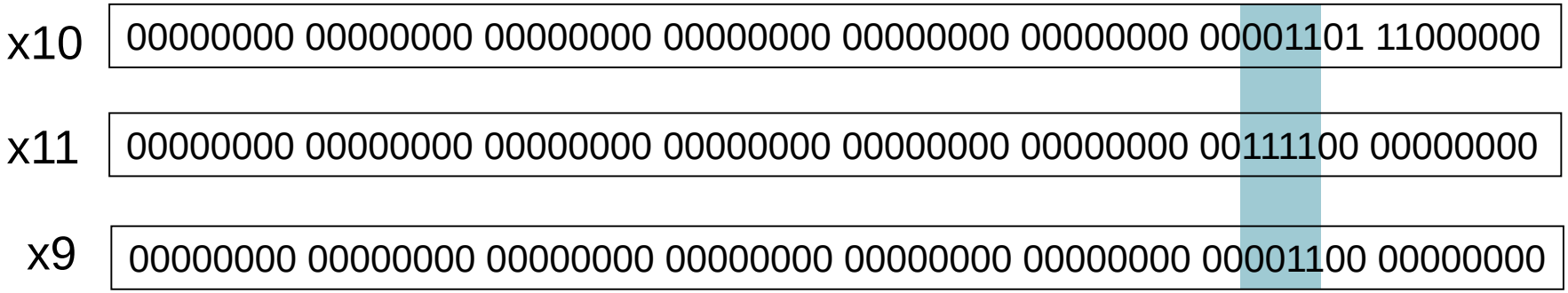
# Shift Operations




- immed: how many positions to shift
- Shift left logical
  - Shift left and fill with 0 bits
  - `slli` by *i* bits multiplies by $2^i$
- Shift right logical
  - Shift right and fill with 0 bits
  - `srli` by *i* bits divides by $2^i$ (unsigned only)

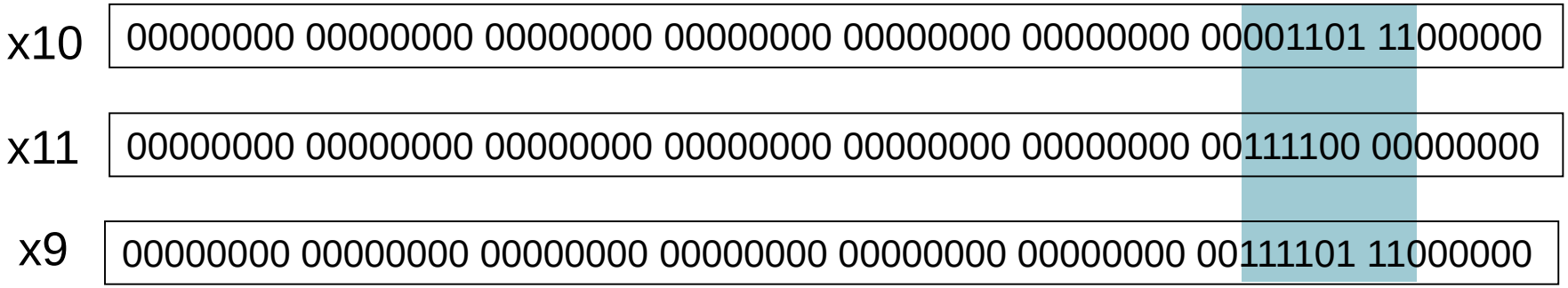
# AND Operations

- Useful to mask bits in a word
- `and x9,x10,x11`



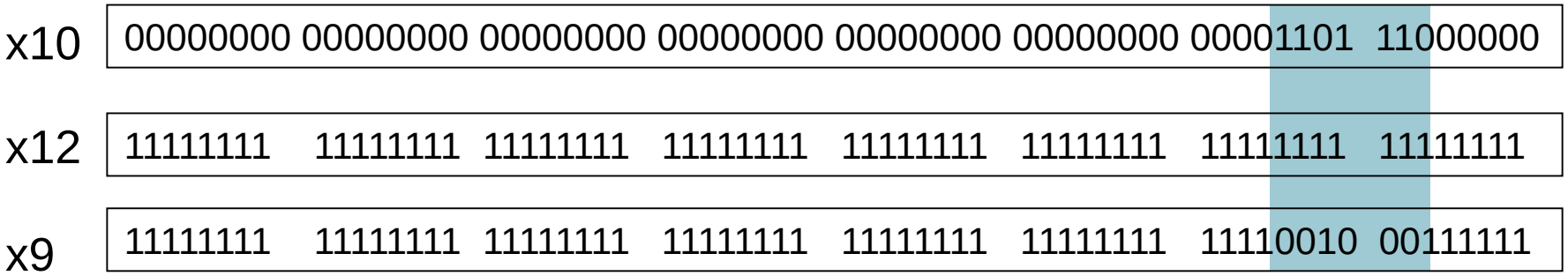
# OR Operations

- Useful to include bits in a word
- `or x9,x10,x11`



# XOR Operations

- Differencing operation
- `xor x9,x10,x12`



# Chapter Outline

- **2 Instructions: Language of the Computer**
  - 2.1 Introduction
  - 2.2 Operations of the Computer Hardware
  - 2.3 Operands of the Computer Hardware
  - 2.4 Signed and Unsigned Numbers
  - 2.5 Representing Instructions in the Computer
  - 2.6 Logical Operations
  - 2.7 Instructions for Making Decisions
  - 2.8 Supporting Procedures in Computer Hardware
  - 2.9 Communicating with People
  - 2.10 RISC-V Addressing for Wide Immediates and Addresses
  - 2.11 Parallelism and Instructions: Synchronization
  - 2.12 Translating and Starting a Program
  - 2.13 A C Sort Example to Put it All Together
  - 2.14 to 2.24 …..

# Instructions for making decisions

- What distinguishes a computer from a simple calculator is its ability to make decisions.
- Otherwise, continue sequentially

- `beq rs1, rs2, L1`
  - if (rs1 == rs2) branch to instruction labeled L1

- `bne rs1, rs2, L1`
  - if (rs1 != rs2) branch to instruction labeled L1

# Conditional Operations

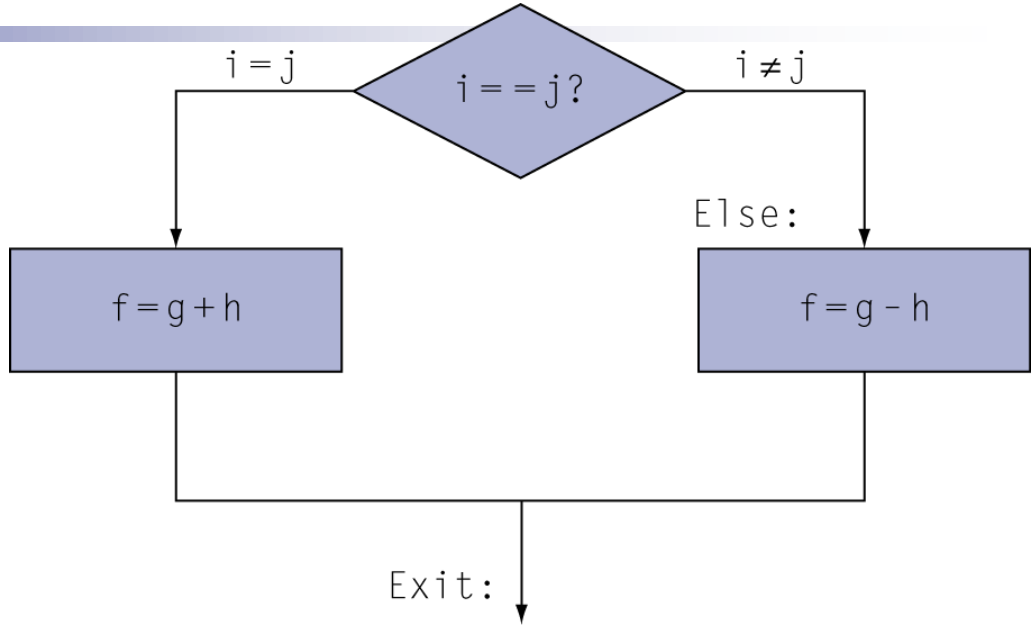
- What distinguishes a computer from a simple calculator is its ability to make decisions.
- Branch to a labeled instruction if a condition is true
  - Otherwise, continue sequentially
- `beq rs1, rs2, L1`
- `bne rs1, rs2, L1`

# Compiling If Statements

- Example

  C code:

```
if (i==j)
    f = g+h;
else
    f = g-h;
```




- f, g, h, i, j in x19, x20, x21, x22, x23
- Compiled RISC-V code?
- HLL better than Assembly as no labels/branches used
- Assembler relieves the compiler/assembly programmer from the tedious task of calculating branch addresses

# Compiling Loop Statements

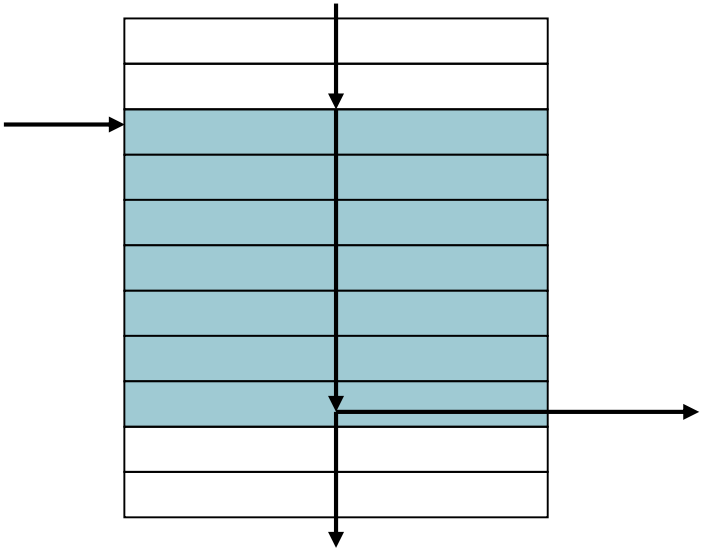
- C code:

```
while (save[i] == k)
    i += 1;
```

- i in x22, k in x24, address of save in x25
- Compiled RISC-V code?

# Basic Blocks

- A basic block is a sequence of instructions with
  - No embedded branches (except at end)
  - No branch targets (except at beginning)



- A compiler identifies basic blocks for optimization
- An advanced processor can accelerate execution of basic blocks

# More Conditional Operations

- `blt rs1, rs2, L1`
- `bge rs1, rs2, L1`
- Example
  - if (a > b)

    a += 1;  // a in x22, b in x23
  - Compiled RISC-V code ?

# Signed vs. Unsigned

- Comparison tests must take into consideration whether the numbers compared are signed or unsigned
- Signed comparison: blt, bge
- Unsigned comparison: bltu, bgeu
- Example
  - x22 = 1111 1111 1111 1111 1111 1111 1111 1111
  - x23 = 0000 0000 0000 0000 0000 0000 0000 0001

# Chapter Outline

- **2 Instructions: Language of the Computer**
  - 2.1 Introduction
  - 2.2 Operations of the Computer Hardware
  - 2.3 Operands of the Computer Hardware
  - 2.4 Signed and Unsigned Numbers
  - 2.5 Representing Instructions in the Computer
  - 2.6 Logical Operations
  - 2.7 Instructions for Making Decisions
  - 2.8 Supporting Procedures in Computer Hardware
  - 2.9 Communicating with People
  - 2.10 RISC-V Addressing for Wide Immediates and Addresses
  - 2.11 Parallelism and Instructions: Synchronization
  - 2.12 Translating and Starting a Program
  - 2.13 A C Sort Example to Put it All Together
  - 2.14 to 2.24 …..

# Procedures

- Procedures in assembly language is one tool programmers use to structure programs for
  - Ease of understanding
  - Code re-reuse
- Similar to *functions/methods* in high level languages like C, Python
- Parameters act as an interface between the procedure and the rest of the program

- Spy analogy – cover the tracks!